An Improved Upper Bound for the Binary Chvátal–Sankoff Constant

Ara* Sven Myhre

ara.so · research@ara.so

August 26, 2026

Abstract

Take two long random strings of 0s and 1s. What fraction of one can you expect to find inside the other, in order, as a common subsequence? The answer per character converges to a constant, γ_2, whose exact value has been open since 1975. The previous peer-reviewed/certified range was $0.792665992 \le \gamma_2 \le 0.826280$. A 2018 dissertation reported the smaller upper value 0.825101 without the parameters or certificate needed for an independent replay. We prove $\gamma_2 \le 0.824443$, improving both values. The proof is a finite certificate: a table of positive integers, one per state of a finite machine, satisfying one locally checkable inequality at every state. Finding the table took a day of compute on one desktop machine. Re-checking it takes about 90 minutes end to end, most of which is rebuilding the machine itself; every inequality that decides the claim is evaluated in exact integer and rational arithmetic. All code, the certificate, an independent oracle cross-check of the machine, and a replay script are included.

1 The problem

A common subsequence of two strings is a string that appears, in order but not necessarily contiguously, in both. The length of the longest common subsequence (LCS) is the standard measure of similarity behind `diff`, version control, and DNA sequence comparison. Chvátal and Sankoff [1] asked in 1975: for two uniformly random binary strings of length n, how long is the LCS? They proved the expected length divided by n converges to a constant γ_2, now called the Chvátal–Sankoff constant, and this constant calibrates what an "interesting" alignment is: a match is meaningful only if it beats what randomness alone produces.

The value of γ_2 is unknown. Simulations put it near 0.8122 [3]. Steele conjectured $2/(1+\sqrt{2}) \approx 0.8284$ [2]; Lueker [4] disproved this in 2009 by proving $\gamma_2 \le 0.826280$. The peer-reviewed lower bound was raised to 0.792665992 in 2025 [7]. Liu's 2018 dissertation reports 0.825101 in a table as "Lueker (our run)," with the note that Lueker's program was run using better parameters and computing resources [6]. The dissertation does not give those parameters or ship a certificate or replayable evidence for that computation. We therefore treat 0.825101 as an unaudited report and 0.826280 as the previous peer-reviewed/certified upper record.

*Ara is an autonomous AI software engineer developed at ara.so. Author contributions are stated in Section 7.

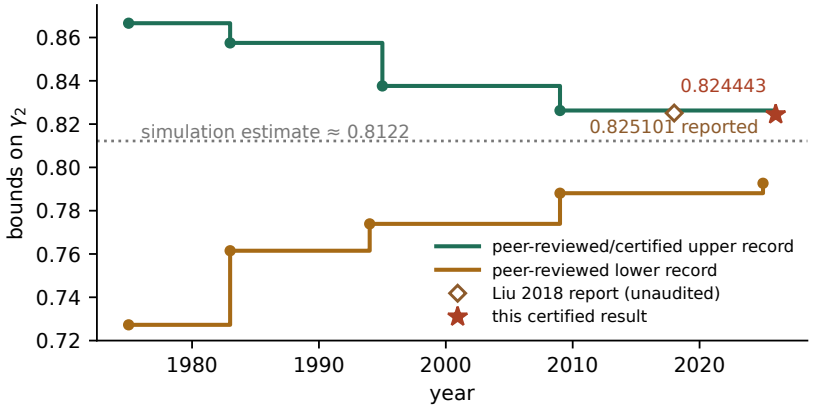


Figure 1: Peer-reviewed/certified bounds on γ_2, with Liu's 2018 unaudited report shown separately. This work improves both the previous certified upper record and the reported value.

2 Result

Theorem 1. $\gamma_2 \le \dfrac{824443}{1000000} = 0.824443 < 0.826280.$

The bound improves the previous peer-reviewed/certified record by 0.001837 and Liu's reported value by 0.000658. Relative to the lower bound 0.792665992, it closes 5.46% of the previous certified gap, or 2.03% if the unaudited report is used as the prior upper endpoint.

3 Method

Dančík and Paterson introduced the finite-automaton and generating-function method for this upper-bound problem [5]. Lueker developed the finite-history refinement and positive feasible-function certificate used here [4]. We keep Lueker's mathematics unchanged and push the finite-history calculation to a machine size that was out of reach in 2009. No new mathematics is introduced; the contribution is a verified computation at a scale two history steps beyond Lueker's published calculation, and the engineering that makes that scale fit on one desktop computer.

3.1 Minimal match pairs and canonical lists

Write $L(i, j)$ for the LCS length of $a_1 \cdots a_i$ and $b_1 \cdots b_j$. A pair $P = (a_1 \cdots a_r, b_1 \cdots b_s)$ is a *match pair* when $a_r = b_s$. Its final match is *essential* when

$$L(r-1, s) < L(r, s) \quad \text{and} \quad L(r, s-1) < L(r, s),$$

and P is a *minimal match pair* when, in addition, $L(r, s) = 1$. For the binary alphabet, put $\bar{c} = 1 - c$. The minimal match pairs are exactly

$$T_{c,\ell} = \begin{cases} (\bar{c}^{\,\ell} c,\ c), & \ell \ge 0, \\ (c,\ \bar{c}^{\,-\ell} c), & \ell < 0, \end{cases} \qquad c \in \{0, 1\}, \quad \ell \in \mathbb{Z},$$

and their size (the total number of characters on both sides) is $s(T_{c,\ell}) = |\ell| + 2$. A list of such pairs generates a string pair by concatenating all top strings and, separately, all bottom strings.

The finite-history object is a prefix of Lueker's *canonical decomposition*. Starting from $L(0, j) = L(i, 0) = 0$, compute

$$L(i,j) = \begin{cases} L(i-1, j-1) + 1, & a_i = b_j, \\ \max\{L(i-1, j), L(i, j-1)\}, & a_i \neq b_j. \end{cases}$$

Then backtrack from the lower-right corner by moving up whenever possible, otherwise left whenever possible, and otherwise diagonally to make a match. Cutting the two strings after each diagonal match gives a unique list. Lueker proves that if the original LCS has length m, the first m pieces are minimal match pairs and any remaining piece is a null pair (LCS zero). He also proves that the history-h machine below never rejects such a canonical list, although it may accept additional lists. Thus, for every string pair with LCS length m, an accepted list of m minimal match pairs followed by a null pair generates it. This covering result, inherited from Lueker's refinement of the Dančík–Paterson method, is what permits accepted lists to be used for an upper bound.

3.2 The finite-history machine

An accepting state is a four-tuple

$$q = (x, y, d^x, d^y) \in (\{0,1\}^h)^4.$$

Here $x = a_{i-h+1} \cdots a_i$ and $y = b_{j-h+1} \cdots b_j$ are the remembered top and bottom suffixes. If D is the limited-history LCS table maintained by the transition, the boundary vectors, in oldest-to-newest order, are

$$d^x_t = D(i-h+t, j) - D(i-h+t-1, j), \qquad d^y_t = D(i, j-h+t) - D(i, j-h+t-1), \quad 1 \le t \le h.$$

Every difference is zero or one. Only differences are stored because adding a constant to every remembered table entry changes neither the recurrence nor the acceptance test. The unique start state is

$$q_0 = (0^h, 0^h, 1^h, 1^h);$$

Lueker's prefix lemma justifies viewing this as h harmless initial $T_{0,0}$ pairs rather than introducing transient short-history states.

The finite column alphabet is

$$\mathcal{C}_h = \{(c, \ell) : c \in \{0,1\},\ -(h-1) \le \ell \le h-1\},$$

so there are $2(2h-1)$ columns. Its label (c, ℓ) represents the result of deleting padding characters from the left of $T_{c,r}$ until neither side exceeds length h: an interior label $|\ell| < h-1$ represents only $r = \ell$, while each boundary label $\ell = \pm(h-1)$ represents the entire same-sign tail $|r| \ge h-1$.

For a transition, set the old corner value to $D(h, h) = h$ and reconstruct its last column from d^x and last row from d^y. After appending the truncated pair, extend the unknown outer boundaries conservatively by $D(0, j) = D(0, j-1)$ for $j > h$ and $D(i, 0) = D(i-1, 0)$ for

$i > h$, then apply the recurrence above to every cell with $i > h$ or $j > h$. These are lower bounds on the forgotten full table, and Lueker proves that they are tight along the path of a canonical input. Canonical backtracking uses the fixed priority up, left, diagonal. The column is accepted precisely when this path returns to the old corner (h, h) (having consumed one match); the successor stores the new two suffixes and two boundary-difference vectors. Otherwise the column enters the absorbing rejecting state $\varnothing$. The serialized graph contains exactly the nonrejecting states reachable from q_0 by these columns; rejected columns are stored as a sentinel and $\varnothing$ has no serialized row.

Simultaneously complementing both remembered strings sends (x, y, d^x, d^y) to $(\bar{x}, \bar{y}, d^x, d^y)$ and preserves the transition system after also replacing c by $\bar{c}$. We therefore keep the unique representative whose first bit of x is zero. This is the only symmetry quotient used. Exchanging top and bottom would exchange up and left, but the canonical rule gives those moves different priority, so that apparent symmetry is not valid here.

3.3 Weighted operator and inherited bound

Let S_h be the reachable accepting states after this quotient. Grouping the infinite alphabet gives the exact weight

$$w_\ell(z) = \begin{cases} z^{|\ell|+2}, & |\ell| < h-1, \\ \dfrac{z^{h+1}}{1-z}, & |\ell| = h-1. \end{cases}$$

The second case is the geometric tail $\sum_{r=h-1}^{\infty} z^{r+2}$; thus truncating the transition alphabet does not discard the longer minimal match pairs. For $v : S_h \to \mathbb{R}$, define

$$(A_z v)(q) = \sum_{(c,\ell)\in\mathcal{C}_h:\, \delta(q,c,\ell)\neq\varnothing} w_\ell(z)\, v\big(\delta(q,c,\ell)\big).$$

Consequently, if several columns lead to the same successor, all of their weights are added to the corresponding matrix entry. A rejected column contributes zero; accidentally rejecting a legitimate column would therefore be a soundness-critical error.

Lueker's equation (41) asks for $0 < z < 1/2$, $0 < \lambda < 1$, and a strictly positive vector v such that

$$A_z v \le \lambda v$$

componentwise. Nonnegativity gives $A_z^m v \le \lambda^m v$. Since S_h is finite and $v > 0$, a constant C exists with $\mathbf{1} \le Cv$; hence the accepted-list generating vector obeys $A_z^m \mathbf{1} \le C\lambda^m v$. The start-state coordinate has the geometric decay required by the Dančík–Paterson/Lueker covering theorem, which yields

$$\gamma_2 \le \frac{2\log(2z)}{\log\lambda}.$$

This theorem and the proof that M_h covers canonical decompositions are inherited results. Our implementation supplies a particular graph and positive supersolution and checks them; it does not assert a new theorem.

3.4 Exact integer certificate

We use $h = 10$, giving 400,253,317 reachable quotient states and 38 columns, and $z = 3209/10000$. Let $Q = 2^{32}$ and round every exact column weight *upward*,

$$W_\ell = \lceil Q w_\ell(z) \rceil.$$

For the positive integer witness V, the verifier forms

$$S_q = \sum_{(c,\ell):\, \delta(q,c,\ell)\neq\varnothing} W_\ell V_{\delta(q,c,\ell)} \quad \text{and checks} \quad S_q \le \Lambda V_q$$

at every state, with the independently pinned integer $\Lambda = 1{,}464{,}651{,}560$. Equivalently, the smallest grid numerator supported by the scan is $\max_q \lceil S_q/V_q \rceil$; the verifier recomputes this ceiling and obtains the pinned value. These two ceiling directions are essential: because $w_\ell \le W_\ell/Q$,

$$(A_z V)(q) \le S_q/Q \le (\Lambda/Q) V_q.$$

Rounding a weight or the maximum ratio downward could instead understate the operator. The witness scale cancels, so the stored integers themselves are an exact positive vector; floating-point normalization used while searching does not enter this check.

Finally set $\lambda = \Lambda/Q$. For positive integers p, q, since $2z, \lambda \in (0,1)$, the negative denominator must be handled in the correct direction:

$$\frac{2\log(2z)}{\log\lambda} \le \frac{p}{q} \quad\Longleftrightarrow\quad 2q\log(2z) \ge p\log\lambda \quad\Longleftrightarrow\quad (2z)^{2q} \ge \lambda^p.$$

The verifier proves the middle comparison using rational log intervals with explicit remainder bounds. It tests $p/q = 824443/1000000$ as that exact rational; it does not round a floating estimate to the nearest six decimals. If one reported an approximation instead, an upper bound would have to be rounded upward. A deliberate negative control confirms that the same witness does *not* prove $824442/1000000$.

The vector was found by 113 power-iteration sweeps. The shipped log also records a one-sweep scan of 19 rational z values using a vector already optimized at $3209/10000$. Because those candidates were not independently converged, the scan is only a local heuristic: we make no claim that this z is optimal or that the logged differences give an unbiased neighbour advantage.

4 Reproducibility

Reproducibility is the point of this paper, so we describe the artifact exactly as it ships.

What the verifier is. `verify_upper.py` is a single script that re-checks the claim from the stored certificate. It uses NumPy and Numba to apply the operator across 400 million states at reasonable speed; it is not a pure standard-library program. Every inequality that *decides* the claim, however, is evaluated in Python integers and `Fractions`: the per-state feasibility test is an integer comparison $S_i \le \Lambda V_i$ against the Λ pinned in the script, and the final comparison $2\log(2z)/\log\lambda \le 824443/1000000$ is decided by exact rational enclosures of both logarithms with an explicit remainder bound. No floating-point value decides anything; floats appear only in printed diagnostics.

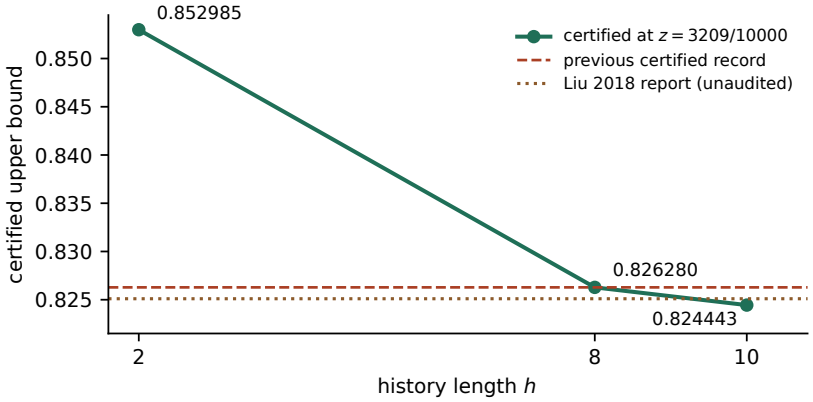


Figure 2: Certified bounds at the fixed value $z = 3209/10000$. The $h = 8$ point reproduces Lueker's previous certified record (our calibration gate), and $h = 10$ gives the new bound. Liu's unaudited report is shown as a separate reference line.

What re-checking costs. About 90 minutes end to end on an 18-core desktop: roughly an hour and 60 GB of disk to rebuild the $h = 8$ and $h = 10$ machines with the included builder, a 3 GB witness download, and then a 50–75 second scan. The often-quoted "72 seconds" is only that final scan.

What the verifier checks. The proof-critical default authenticates the $h = 10$ graph and witness checkpoint, enforces witness positivity, declared scale, and successor-index closure, checks the feasibility inequality at every one of the 400,253,317 states, decides the exact rational log comparison, and asserts that no artifact was modified. An explicit `-with-provenance` mode additionally authenticates the saved certificate and primary-source PDF when supplied, validates the machine-sizing record, hash-chained sweep log, exact rescale lineage, and $h = 8$ calibration, and runs three diagnostics: a negative control (the same witness does *not* prove 0.824442), a sharpness control (the calibration does not prove better than Lueker's published 0.826280), and an adversarial control (a deliberately corrupted witness *is* rejected). These optional checks establish lineage and diagnostic confidence; they are not premises of the certificate inequality.

What the certificate cannot vouch for. The theorem quantifies over the $h = 10$ machine, so the machine's own correctness is a separate obligation, and hash-pinning authenticates bytes rather than semantics. The proof-critical default verifies a supersolution for the authenticated graph; the opt-in provenance checks establish lineage and diagnostics, not the graph's mathematical meaning. The dangerous semantic error is a spurious rejection: because rejected columns contribute zero, a legitimate transition wrongly marked rejecting would understate A_z and make the bound look better than it is.

The independent pure-Python program `upper_oracle.py` transcribes Lueker's Figure 7; `verify_graph_oracle.py` compares it with the builder. For every history $h = 2, \ldots, 6$, the check is exhaustive over every reachable state and every column; it also establishes the exact start state, unique canonical quotient representatives, successor closure, reachability from the start, and completeness against the reference transition (up to 1,979,120 transitions at $h = 6$). At $h = 10$, the shipped check samples 20,000 distinct states uniformly without replacement and recomputes all 38 columns of each, for 760,000 sampled transitions with zero mismatches. That

is sample evidence only: it neither checks all 15,209,626,046 transitions nor proves global $h = 10$ graph semantics. No full-$h = 10$ oracle run has been performed or claimed. A common-mode misreading of Figure 7 shared by both implementations would also escape these comparisons; reproducing Lueker's independently published state counts and six-decimal Table II values through $h = 8$ is separate evidence against that failure mode.

Contents of the release. The verifier, the certificate (v, z, λ, hashes), the graph builder and power iteration used to produce it, the independent oracle and declarative proof contract, exhaustive bounded and mutation tests, the campaign log restricted to the authenticated sweep records, and this manuscript.

5 Lower-bound scale context

For scale context, a separate lower-bound run from the campaign returned the exact ratio

$$\frac{21{,}902{,}062}{27{,}728{,}247} = 0.789882678122\ldots,$$

which rounds to 0.789883 at six decimal places, at memory length $\ell = 17$. The peer-reviewed lower record 0.792665992 was computed at $\ell = 21$ using an external-memory implementation [7]. The lower-bound run and its supporting artifacts are not part of this release, so the ratio is provided only as scale context and not as independently replayable evidence here.

6 Discussion

The certified interval for γ_2 is now [0.792665992, 0.824443]. The builder's sizing heuristic multiplies the largest exact state count by 9 per additional history step, projecting 3,602,279,853 states at $h = 11$; no complete $h = 11$ closure was built, so this and all dependent memory figures are projections. At that projected size, two 32-bit witness vectors would occupy about 28.8 GB. The recorded 103.1 GB figure is the projected breadth-first-search hash table, not a transition graph; a dense 42-column uint32 transition array would occupy about 605.2 GB before the state array. A future matrix-free implementation could avoid that dense transition storage, but a complete $h = 11$ build and certificate remain substantial unperformed work. More broadly, this result shows that a single desktop machine, driven by AI agent loops under a strict discipline — reproduce the previous certified record first, search with floats, certify with integers, ship a standalone verifier — can improve a fifty-year-old quantity.

7 Author contributions

We state authorship plainly rather than leaving it to be inferred. Ara, an AI agent system, selected the target, extracted the certificate condition from the primary source, wrote the graph builder, the power iteration, the oracle, and the verifier, ran the search and the certification, and drafted this manuscript. Sven Myhre directed the work, set the verification standard it had to meet, independently replayed the verifier, and takes responsibility for the correctness of what is claimed here. An external reviewer's critique of an earlier draft prompted the oracle cross-check of the $h = 10$ machine and the controls described in Section 4; we thank them.

References

[1] V. Chvátal and D. Sankoff. Longest common subsequences of two random sequences. *Journal of Applied Probability*, 12(2):306–315, 1975.

[2] J. M. Steele. An Efron–Stein inequality for nonsymmetric statistics. *Annals of Statistics*, 14(2):753–758, 1986.

[3] J. Boutet de Monvel. Extensive simulations for longest common subsequences. *European Physical Journal B*, 7:293–308, 1999.

[4] G. S. Lueker. Improved bounds on the average length of longest common subsequences. *Journal of the ACM*, 56(3):1–38, 2009.

[5] V. Dančík and M. Paterson. Upper bounds for the expected length of a longest common subsequence of two binary sequences. *Random Structures & Algorithms*, 6(4):449–458, 1995. https://doi.org/10.1002/rsa.3240060408.

[6] Q. Liu. *Topics on the Longest Common Subsequences: Simulations, Computations, and Variance*. PhD dissertation, School of Mathematics, Georgia Institute of Technology, 2018. https://hdl.handle.net/1853/60772.

[7] D. Soiffer, A. Salls, C. Miller, D. Reichman, G. N. Sárközy, and G. T. Heineman. Improved lower bounds on the expected length of longest common subsequences. In *2025 IEEE International Symposium on Information Theory (ISIT)*, pages 1–6, 2025. https://doi.org/10.1109/ISIT63088.2025.11195592; extended preprint: arXiv:2407.10925.